



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Game Programming

Fachhochschule Oberösterreich

Dr. Andreas Riegler

Prof. of Interactive Systems
Digital Media Department

Research Groups

- Digital Media Lab
- Human-Computer Interaction Group
- Mobile Interactive Systems
- Human Interfaces & Virtual Environments

CS PhD @ Johannes Kepler University Linz



Agenda

-
1. Introduction to Game Programming
 2. Course Grading

Game Programming

Course Mode

→ Lectures (VO)

- Advanced Programming Concepts
- Game Design Patterns
- Shaders & Materials
- Audio: Sound & Music

→ Practical Work (UE)

- 5 Exercises in groups (systematic building & evaluating a game, including your ideas)
 - Groups according to Game Production Module
- Final game must include
 - Advanced programming concepts and software design patterns
 - Custom shader & materials
 - Custom audio

→ Lecturers: Andreas Riegler (VO), Rui Kremsmayr (UE), Samuel Zuehlke (UE)

Course Grading

Game Programming

Course Grading

→ Lectures (VO)

- Digital Exam at end of semester
- Based on lecture slides

→ Practical Work (UE)

- In-Class Exercises (graded by tutor and UE lecturers)



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Game Programming

Game Programming

Introduction

- A **project** contains assets, packages, project settings and scenes
- A **scene** contains GameObjects arranged in a hierarchy
- **GameObjects** are empty containers until components define behavior or representation
- The **Inspector** exposes serialized fields and component settings
- The **Play Mode** loop runs the scene using Unity's frame and physics update cycles

Game Programming

C# Script Anatomy in Unity

- A Unity script is typically a class derived from **MonoBehaviour**
- The class name and file name should match
- Unity calls specific methods such as Start and Update automatically
- Public or serialized fields can be configured in the Inspector

```
using UnityEngine;

public class Rotator : MonoBehaviour
{
    [SerializeField]
    private float speed = 90f;

    private void Update()
    {
        transform.Rotate(0f, speed * Time.deltaTime, 0f);
    }
}
```

Game Programming

Variables

- Fields store state across frames
- Local variables exist only during a method call
- Prefer **private fields with [SerializeField]** over public fields for editable configuration
- Initialize safe defaults, but tune gameplay values in the Inspector
- Use meaningful names that describe game semantics, not only data types

```
public class Health : MonoBehaviour
{
    [SerializeField] private int maxHealth = 100;
    [SerializeField] private bool invulnerable;

    private int currentHealth;

    private void Start()
    {
        currentHealth = maxHealth;
    }
}
```

Game Programming

Methods

- Methods encapsulate actions or queries
- A method should usually do one conceptually coherent task
- Parameters pass information into a method; return values pass information out
- Method names should read like verbs for commands and nouns/adjectives for queries

```
private bool IsAlive()
{
    return currentHealth > 0;
}

private void TakeDamage(int amount)
{
    if (invulnerable || !IsAlive()) return;

    currentHealth -= amount;
    currentHealth = Mathf.Max(currentHealth, 0);
}
```

Game Programming

Control Flow: Decisions and Loops

- if statements express conditional behavior: only execute when a condition is true
- switch statements are useful for discrete gameplay states
- Loops repeat logic, but avoid heavy loops in Update unless bounded and necessary
- In games, many loops are implicit: Unity already calls Update every frame

```
private enum EnemyState { Idle, Patrol, Chase }  
private EnemyState state;  
  
private void Update()  
{  
    switch (state)  
    {  
        case EnemyState.Idle:    LookForPlayer(); break;  
        case EnemyState.Patrol:  FollowPath(); break;  
        case EnemyState.Chase:   ChasePlayer(); break;  
    }  
}
```

Game Programming

Classes

- A class defines a type: data plus behavior
- An object is an instance of a type at runtime
- In Unity, MonoBehaviour objects are component instances attached to GameObjects
- Separate reusable logic from scene-specific configuration when possible

```
1 public class ScoreCounter
2 {
3     public int Score { get; private set; }
4
5     public void AddPoints(int points)
6     {
7         Score += points;
8     }
9 }
10
11 // MonoBehaviour components can use ordinary C# classes
12 private ScoreCounter score = new ScoreCounter();
```

Game Programming

Value Types vs Reference Types

- int, float, bool, Vector2 and Vector3 behave like values: assignment copies data
- Classes and UnityEngine.Object **references point to existing objects**
- Null reference errors happen when a reference variable does not point to an object
- Check required references early, especially when they are assigned in the Inspector

```
[SerializeField]
private Transform target;

private void Start()
{
    if (target == null)
    {
        Debug.LogError("Target is missing", this);
        enabled = false;
    }
}
```

Game Programming

Collections

- Arrays have fixed size and are efficient for stable collections
- List<T> grows dynamically and is common for runtime groups of objects
- Dictionary<TKey,TValue> maps keys to values for fast lookup
- Choose the collection that matches how the data is accessed

```
using System.Collections.Generic;

[SerializeField] private List<GameObject> spawnPoints;
private Dictionary<string, int> inventory = new();

private Transform RandomSpawnPoint()
{
    int index = Random.Range(0, spawnPoints.Count);
    return spawnPoints[index].transform;
}
```

Game Programming

Namespaces

- Namespaces organize code and reduce naming conflicts
- *using UnityEngine* gives access to Unity types such as `GameObject` and `Vector3`
- Keep project code in your own namespace for larger games or teaching projects
- Avoid placing unrelated systems into one namespace just because they are in one folder

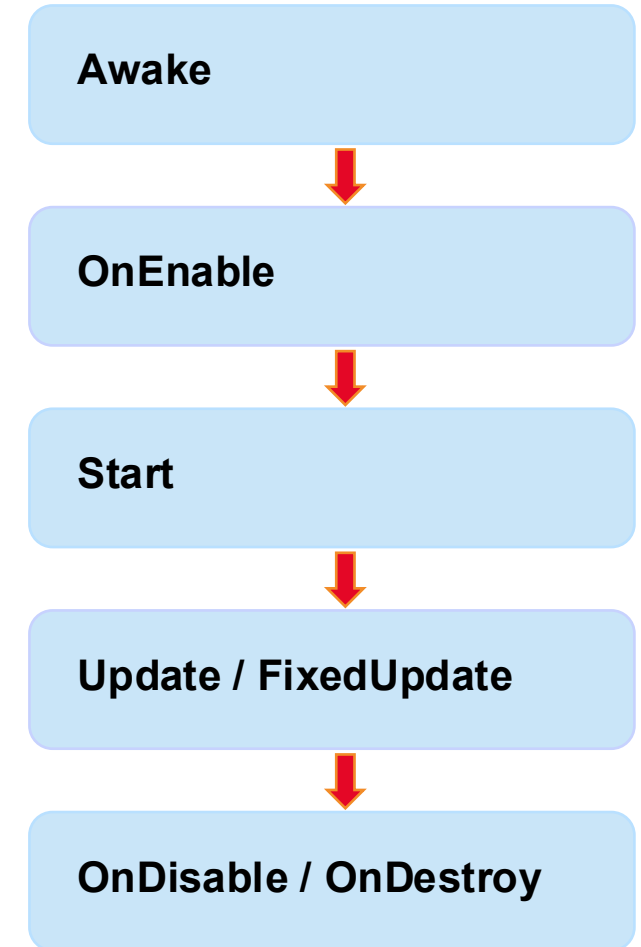
```
using UnityEngine;

namespace MTD.GameProgramming
{
    public class Coin : MonoBehaviour
    {
        [SerializeField] private int value = 1;
        public int Value => value;
    }
}
```


Game Programming

MonoBehaviour Lifecycle

- Unity calls lifecycle methods at specific moments in an object's existence
- Awake is used for internal initialization; Start occurs before the first frame update
- Update runs once per rendered frame; FixedUpdate runs on the physics timestep
- OnEnable and OnDisable are useful for registering and unregistering events
- Do not invent your own lifecycle names and expect Unity to call them



Game Programming

Introduction

- Awake: set up references and internal data
- Start: initialize behavior that may depend on other objects already being awake
- Update: read input, drive non-physics logic and perform per-frame behavior
- Keep Update small; delegate complex work to named methods

```
private Rigidbody rb;

private void Awake()
{
    rb = GetComponent<Rigidbody>();
}

private void Start()
{
    Debug.Log("Player ready");
}

private void Update()
{
    ReadInput();
}
```

Game Programming

Time.deltaTime: Frame-rate Independent Motion

- Update is called once per frame, but frame duration is not constant
- Multiplying by Time.deltaTime converts units from “per frame” to “per second”
- Use deltaTime for movement, timers and continuous interpolation in Update
- Physics code in FixedUpdate usually uses Time.fixedDeltaTime implicitly through physics forces

```
[SerializeField] private float speed = 5f;

private void Update()
{
    float x = Input.GetAxisRaw("Horizontal");
    Vector3 movement = new Vector3(x, 0f, 0f);

    transform.position += movement * speed * Time.deltaTime;
}
```

Game Programming

GameObject

- A GameObject has a name, active state, tag, layer and Transform
- It becomes meaningful through attached components: MeshRenderer, Collider, Rigidbody, scripts and more
- Use SetActive for enabling or disabling entire object trees
- Prefer references assigned in the Inspector over expensive repeated scene searches
- GameObject.Find can be useful for quick prototypes but is fragile in production code

Mental Model

GameObject = identity + hierarchy
Component = capability
Transform = position/rotation/scale
Prefab = reusable template

Game Programming

Components

- “Composition over Inheritance”
- Unity favors composition: build behavior by combining components
- GetComponent<T>() retrieves another component on the same GameObject
- TryGetComponent avoids exceptions and expresses optional dependencies
- RequireComponent can enforce required dependencies at edit time

```
[RequireComponent(typeof(Rigidbody))]  
public class Jumper : MonoBehaviour  
{  
    private Rigidbody rb;  
  
    private void Awake()  
    {  
        rb = GetComponent<Rigidbody>();  
    }  
  
    public void Jump(float force)  
    {  
        rb.AddForce(Vector3.up * force, ForceMode.Impulse);  
    }  
}
```

Game Programming

Transform

- Every GameObject has exactly one Transform
- Transform defines local position, local rotation and local scale relative to the parent
- World-space properties describe the final transform after hierarchy composition
- Parents are useful for grouping, but scaling parents can produce unexpected child behavior

```
private void MoveTo(Transform target)
{
    transform.position = target.position;
}

private void Face(Transform target)
{
    Vector3 direction = target.position - transform.position;
    transform.forward = direction.normalized;
}
```

Game Programming

Vector2 & Vector3

- Vector2 represents 2D quantities: screen input, top-down movement, UI positions
- Vector3 represents 3D quantities: world positions, directions and velocities
- Common operations: magnitude, normalized, distance, dot product and cross product
- A direction is not a position; naming should make that distinction explicit

```
Vector3 playerPosition = transform.position;  
Vector3 targetPosition = target.position;  
  
Vector3 toTarget = targetPosition - playerPosition;  
float distance = toTarget.magnitude;  
Vector3 direction = toTarget.normalized;  
  
transform.position += direction * speed * Time.deltaTime;
```

Game Programming

Movement

- Read input as an abstract intention, not as direct world mutation
- Normalize diagonal movement to avoid faster diagonal speed
- Multiply by speed and deltaTime for continuous movement
- For physics-controlled objects, move through Rigidbody in FixedUpdate

```
private Vector3 moveInput;

private void Update()
{
    float x = Input.GetAxisRaw("Horizontal");
    float z = Input.GetAxisRaw("Vertical");
    moveInput = new Vector3(x, 0f, z).normalized;
}

private void FixedUpdate()
{
    rb.MovePosition(rb.position + moveInput * speed * Time.fixedDeltaTime);
}
```


Game Programming

Input

- The classic Unity Input API is simple and appears in many examples, but it is string-based
- The newer Input System uses actions, devices and generated C# wrappers
- Teaching strategy: learn the movement logic first, then replace the input source
- Keep input reading separate from game actions when possible
- Do not hide core gameplay rules inside UI or device-specific input code

Design Separation

Input source
Intent / command
Gameplay action
World response
Feedback

Game Programming

Collisions & Triggers

- Collisions involve physical contact; triggers detect overlap without solid collision response
- At least one object usually needs a Rigidbody for physics callbacks to fire reliably
- Use tags, layers or component checks to identify what was hit
- Prefer component checks over tag strings when behavior matters

```
private void OnTriggerEnter(Collider other)
{
    if (other.TryGetComponent<PlayerStats>(out var player))
    {
        player.AddCoins(value);
        Destroy(gameObject);
    }
}

private void OnCollisionEnter(Collision collision)
{
    Debug.Log("Hit: " + collision.gameObject.name);
}
```

Game Programming

Prefabs & Instantiate

- A prefab is a **reusable asset template** for GameObjects and component configuration
- Instantiate creates a runtime copy of a prefab
- Keep references to spawned objects when you need to configure or destroy them later
- Spawning is simple; uncontrolled spawning is a performance and design problem

```
[SerializeField] private GameObject enemyPrefab;  
[SerializeField] private Transform spawnPoint;  
  
public void SpawnEnemy()  
{  
    GameObject enemy = Instantiate(  
        enemyPrefab,  
        spawnPoint.position,  
        spawnPoint.rotation);  
  
    enemy.name = "Enemy_Runtime";  
}
```

Game Programming

Destroy

- Destroy marks a Unity object for **removal** at the end of the frame
- Destroying the GameObject removes all its components and children
- Use object pooling for frequently created and destroyed objects such as bullets
- OnDestroy is for cleanup; do not rely on it for normal gameplay sequencing

```
[SerializeField] private float lifetime = 3f;

private void Start()
{
    Destroy(gameObject, lifetime);
}

private void OnDestroy()
{
    Debug.Log("Projectile removed");
}
```

Game Programming

Camera

- The camera defines what the player sees; its transform is part of gameplay feel
- A follow camera can be implemented by copying or smoothing toward a target position
- Camera-relative movement uses the camera's forward and right vectors projected onto the ground plane
- Avoid mixing camera logic into the player controller when the camera needs independent tuning
- Cinemachine is often preferred for production camera behavior, but the vector math remains relevant

Game Programming

Quaternion

- Unity stores rotations as Quaternions internally
- Use Quaternion.identity for no rotation
- Use Quaternion.LookRotation to face a direction
- Use Quaternion.Slerp or RotateTowards for smooth rotation
- Avoid editing x/y/z/w Quaternion fields directly

```
Vector3 direction = target.position - transform.position;  
direction.y = 0f;  
  
if (direction.sqrMagnitude > 0.001f)  
{  
    Quaternion targetRotation = Quaternion.LookRotation(direction);  
    transform.rotation = Quaternion.RotateTowards(  
        transform.rotation,  
        targetRotation,  
        turnSpeed * Time.deltaTime);  
}
```

Game Programming

Rigidbody

- A Rigidbody lets Unity's physics engine control motion
- Move physics objects in FixedUpdate, not Update
- Use forces for physically motivated motion; use MovePosition for kinematic-style motion
- Do not fight physics by setting transform.position every frame on dynamic bodies

```
private void FixedUpdate()
{
    Vector3 force = moveInput * acceleration;
    rb.AddForce(force, ForceMode.Acceleration);
}

public void Teleport(Vector3 position)
{
    rb.position = position;
    rb.velocity = Vector3.zero;
}
```

Game Programming

UI

- Connect interface and gameplay
- UI scripts often translate game state into visual feedback
- Keep UI display code separate from the systems that own the game state
- Assign TextMeshPro or UI references in the Inspector
- Update UI only when state changes where possible, not every frame by default

```
using TMPro;

public class ScoreView : MonoBehaviour
{
    [SerializeField] private TMP_Text scoreLabel;

    public void SetScore(int score)
    {
        scoreLabel.text = $"Score: {score}";
    }
}
```


Game Programming

Audio

- AudioSource plays clips in the scene or attached to objects
- One-shot sounds are useful for UI clicks, impacts and pickups
- Audio is gameplay feedback: it confirms actions, warns players and supports rhythm
- Avoid creating a new AudioSource for every tiny sound; plan an audio manager or pooling strategy

```
[SerializeField] private AudioSource source;  
[SerializeField] private AudioClip pickupClip;  
  
private void OnTriggerEnter(Collider other)  
{  
    if (!other.CompareTag("Player")) return;  
  
    source.PlayOneShot(pickupClip);  
    gameObject.SetActive(false);  
}
```

Game Programming

Coroutines: Time-Based Sequences

- A coroutine can **pause execution and resume later** without blocking the whole game
- *StartCoroutine* begins an IEnumerator method
- *yield return new WaitForSeconds(seconds)* waits in scaled game time
- Coroutines are excellent for delays, fades, cooldowns and scripted sequences

```
private IEnumerator Blink(float duration)
{
    renderer.enabled = false;
    yield return new WaitForSeconds(duration);
    renderer.enabled = true;
}

public void StartBlink()
{
    StartCoroutine(Blink(0.25f));
}
```

Game Programming

Coroutines: Patterns and Pitfalls

- Store a Coroutine handle if you need to stop a running routine
- Avoid starting duplicate coroutines accidentally every frame
- Use `WaitForSecondsRealtime` for UI or pause-menu behavior unaffected by `Time.timeScale`
- Coroutines are not threads; they still run on the main Unity thread

```
private Coroutine reloadRoutine;

public void Reload()
{
    if (reloadRoutine != null) return;
    reloadRoutine = StartCoroutine(ReloadRoutine());
}

private IEnumerator ReloadRoutine()
{
    yield return new WaitForSeconds(reloadTime);
    ammo = maxAmmo;
    reloadRoutine = null;
}
```

Game Programming

ScriptableObjects: Data as Assets

- ScriptableObjects **store reusable data** outside scene objects
- They are useful for items, enemies, abilities, tuning parameters and event channels
- They reduce duplication and make balancing easier

```
[CreateAssetMenu(menuName = "Game/Weapon")]  
public class WeaponDefinition : ScriptableObject  
{  
    public string displayName;  
    public int damage;  
    public float cooldown;  
    public AudioClip fireSound;  
}
```

Game Programming

Serialization

- Unity serializes public fields and private fields marked with `[SerializeField]`
- Serialized data is stored in scenes, prefabs and assets
- Note: Renaming serialized fields can lose Inspector values

```
public class Door : MonoBehaviour
{
    [SerializeField] private bool startsOpen;
    [SerializeField] private float openSpeed = 2f;
}
```

Game Programming

Key Takeaways

- Unity programming combines general C# competence with engine-specific execution rules
- GameObjects are containers; components provide behavior; Transform defines spatial relationships
- Vector and Quaternion literacy is essential for movement, camera logic and interaction
- Coroutines express time-based sequences without blocking the game loop
- Advanced course topics build naturally on these basics: events, lambdas, shaders, audio and profiling

Thanks for your Attention



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at